



Universidad
Indoamérica

ユニット 6・オペレーティングシステムの基礎

シグナルと共有メモリ

Universidad Tecnológica Indoamérica 演習 TD5

シグナル：プロセスへの非同期の通知

カーネル

SIGSEGV（不正なアクセス）、SIGBUS、SIGPIPE（読み手のいないパイプ）、SIGCHLD（子が終了）、SIGALRM

端末

Ctrl-C → SIGINT、Ctrl-\ → SIGQUIT、Ctrl-Z → SIGTSTP、閉じる → SIGHUP

別のプロセス

`kill(pid, sig)` と `kill` コマンド：
SIGTERM、SIGKILL、SIGUSR1...

シグナルはプロセスの状態を変える：終了させる、停止させる、再開させる...あるいはその関数の1つを実行させる。

非同期の送信、保留中のシグナル、そして捕捉されるタイミング

- 送信は、相手が実行中かどうかに関係なく、いつでも起こりうる
- **保留**：受け取ったが、まだ配送されていない（ブロック中か、カーネルがまだ確かめていない）
- **ユーザモードに戻るとき**に配送される：システムコールやコンテキストスイッチの後

元の内容への補足（「シグナルのキューは存在しない」）：標準シグナルについては正しい（保留中のものは1ビットで、重複は失われる）が、**リアルタイムシグナル**（SIGRTMIN ... SIGRTMAX）はキューに入り、値を運べる（sigqueue）。

kill とよく使うシグナル

C

```
int kill(pid_t pid, int sig);  
/* pid > 0: that process · 0: my process group  
   < -1: group -pid · -1: every process I may signal  
   errors: EINVAL, EPERM, ESRCH */
```

元の内容の修正（スライド 97）：SIGHUP、SIGINT、SIGTSTP...は「マスクもブロックもできない」とあった。捕捉も無視もブロックもできないのは **SIGKILL** と **SIGSTOP** だけで、ほかはすべてできる。

シグナル	既定
SIGHUP · SIGINT · SIGTERM	終了
SIGQUIT · SIGSEGV · SIGBUS	終了 + core
SIGKILL	終了（常に）
SIGPIPE · SIGALRM · SIGUSR1 · SIGUSR2	終了
SIGSTOP · SIGTSTP	停止
SIGCONT	再開
SIGCHLD	無視

保留、ブロック、配送

シグナル	n	保留	ブロック	動作	既定
SIGINT (Ctrl-C)	2	0	—	SIG_DFL	終了
SIGQUIT (Ctrl-\)	3	0	—	SIG_DFL	終了 + core
SIGKILL	9	0	不可	SIG_DFL	終了
SIGUSR1	10	1	はい	ハンドラ	終了
SIGSEGV	11	0	—	SIG_DFL	終了 + core
SIGPIPE	13	0	—	SIG_DFL	終了
SIGALRM	14	0	—	SIG_DFL	終了
SIGCHLD	17	0	—	SIG_DFL	無視
SIGCONT	18	0	—	SIG_DFL	再開
SIGSTOP	19	0	不可	SIG_DFL	停止
SIGTSTP (Ctrl-Z)	20	0	—	SIG_DFL	停止
SIGRTMIN	34	0	—	SIG_DFL	終了

プロセス

pause / sigsuspend で睡眠中

起きたこと

- ・ sigprocmask(SIG_BLOCK, {SIGUSR1}); while (!flag)...
- ・ kill(pid, SIGUSR1) : カーネルが SIGUSR1 を保留にする
- ・ 最悪の瞬間に届くが、保留のまま（ブロック中）
- ・ sigsuspend(SIGUSR1を除いたマスク) : 解除と待機を「1つの」操作で行う

ハンドラ： `signal` から `sigaction` へ

```
C

sighandler_t signal(int sig, sighandler_t f); /* obsolete; SIG_ERR on error */
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *old);
struct sigaction {
    void (*sa_handler)(int); /* or SIG_DFL, SIG_IGN */
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; /* blocked during the handler */
    int sa_flags; /* SA_RESTART SA_SIGINFO SA_NOCLDSTOP... */
};
```

- ハンドラの実行中、同じシグナルはブロックされたまま
- 戻ると、プログラムは元の場所から再開する。`sa_flags`：`SA_RESTART` は中断された呼び出しをやり直す（さもないと `EINTR`、ユニット 2）。`SA_SIGINFO` は `sa_sigaction` を使う。`SA_NOCLDSTOP`：`SIGCHLD` は子が終了したときだけで、停止や再開では送られない
- 中では**非同期シグナル安全**な関数（`write` は可、`printf` と `malloc` は不可）と `volatile sig_atomic_t` の変数だけを使う
- `signal()` はシステムによって意味が違う：`sigaction` を使う

待つ・予約する： `pause`、`alarm`、`sleep`

- `pause()` : シグナルが終了させるかハンドラを実行するまで眠る。常に `-1` を返す
- `alarm(s)` : `s` 秒後に `SIGALRM` (既定では終了)
- `sleep(s)` : `s` 秒間停止する

更新 (スライド 101) : 元は `sleep` が `pause` + `alarm` で実装されており、混ぜて使ってはいけなかった。glibc/Linux の `sleep` は `clock_nanosleep` を使い、アラームには触れない。今は `nanosleep` や、`timer_create` ・ `timerfd` のタイマが好まれる。

元の例...とその隠れたバグ

C

```
/* slide 102: mysleep */
signal(SIGALRM, do_nothing);
alarm(s);
pause();      /* what if SIGALRM arrives BEFORE pause? */

/* slide 103: ping-pong, parent and child after fork() */
kill(getppid(), SIGUSR1);
pause();      /* same race: the answer may come first */
```

- 準備と `pause()` の間にシグナルが届くと、ハンドラが先に実行され、`pause` は永遠に待つ (スライド 105)
- 見つけるのも再現するのも難しいバグ：スケジューラ次第 (ユニット 5 の `mypipe` の競合と同じ)
- 元の結論 (スライド 104)：同期に `signal()` / `pause()` を使わない

ブロック、確認、 sigsuspend

C tds/td5/mysignal.c

```
/* Block SIGUSR1 outside sigsuspend: a signal arriving
   and the wait stays pending instead of being lost (th
sigemptyset(&bblock);
sigaddset(&bblock, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &bblock, &wait_mask) == -1);
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unbl
```

sigprocmask がブロックし、 sigsuspend が
ブロック解除と睡眠を1つのアトミックな操作で
行う。集合の操作： sigemptyset、 sigfillset、
sigaddset、 sigdelset、 sigismember。

実際の出力

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep
0.2; kill -STOP $P; kill -USR1 $P; sleep 0.2; kill -CONT
$P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status:
$? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1
./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

シグナル + メモリ：送り手と受け手

実際の出力

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

- 受け手はマップしたファイルの先頭に自分の PID を書き、`sigsuspend` で SIGUSR1 を待つ
- 送り手は PID を読み、同じマップ済みファイルにメッセージを書いてシグナルを送る：シグナルは**知らせ**、メモリは**運ぶ**

今はほかにも：`signalfd`（シグナルをディスクリプタからデータとして読む）、`pidfd_send_signal`（PID の再利用による競合なしにシグナルを送る）。

プロセス間でメモリを共有する

- 仮想メモリは隔離する：各プロセスが自分のページを持つ（ユニット 4）
- 共有 = **2つのプロセス**の仮想ページが**同じ物理フレーム**を指すようにすること
- **待ち合わせ場所**が必要：ファイルか名前

`mmap` + ファイル

ファイルが待ち合わせ場所であり、データも保存する（残る）

`shm_open` +
`mmap`

POSIX、`/dev/shm` 内の名前
(RAM 上)、今の推奨

`ftok` + `shmget`
+ `shmat`

System V、数値のキー。ファイルの中身は使わない

ファイルをメモリにマップする： `mmap`

C

```
void *mmap(void *addr, size_t len, int prot, int flags,  
           int fd, off_t offset); /* MAP_FAILED on error */  
int munmap(void *addr, size_t len);  
int msync(void *addr, size_t len, int flags);
```

- `prot` : `PROT_NONE`、`PROT_READ`、`PROT_WRITE`、`PROT_EXEC`
- `flags` : `MAP_SHARED`、`MAP_PRIVATE` (コピーオンライト)、`MAP_ANONYMOUS`
- エラー : `EACCES` (通常ファイルでない、またはディスクリプタのモードが要求を許さない : `O_WRONLY` と `O_APPEND` は使えない)、`ENOMEM`

元の内容の修正 (スライド 118) : `len` はページサイズの倍数である必要はない (切り上げられる)。

倍数でなければならないのは `offset` 。また、ファイルの終わりより先にアクセスすると `SIGSEGV` ではなく **`SIGBUS`** になる。次のアニメーションが実際の出力で見せる。

ファイルを通してメモリを共有する

C tds/td5/sender.c

```
int fd = open(FILENAME, O_RDWR);
exit_if(fd == -1, "open " FILENAME);
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");

...

shared[off] = '\\0';

exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also push to the file */
exit_if(kill(pid, SIGUSR1) == -1, "kill");
exit_if(munmap(shared, SIZE) == -1, "munmap");
```

- `O_RDWR` : `PROT_WRITE` + `MAP_SHARED` には読み書き用のディスクリプタが必要
- `mmap` の後はディスクリプタを閉じてよい：マッピングは生きたまま
- `shared[i]` に書くことはファイルに書くこと：同じファイルをマップしている受け手にも見える
- `munmap` はマッピングを外し、`msync` はディスクへの書き出しを強制する

mmap の限界：SIGBUS と PROT_*

ページ 0 (0 ... 4095)

ファイルより先の読み書きできる領域：0 として読め、保存されない

ファイル：128 B

map[i] = 'x' で書いたバイト

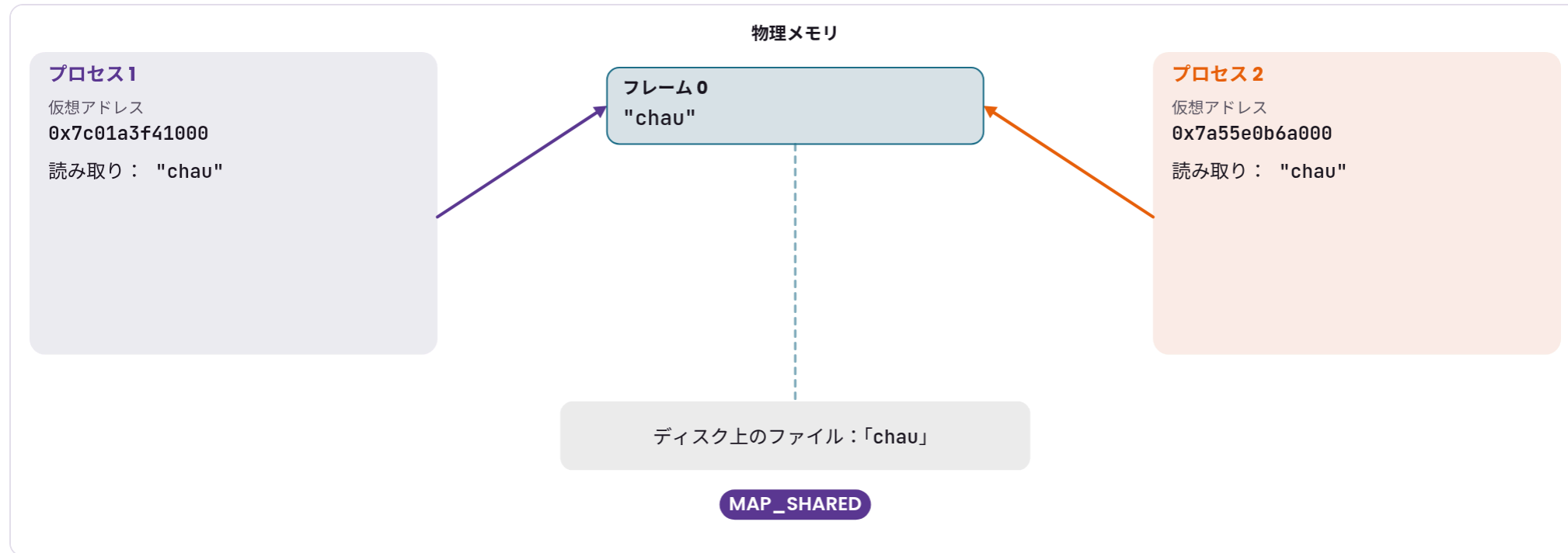
ページ 1 (4096 ... 8191)

裏にファイルなし → SIGBUS

SIGBUS @ 4096

その後のファイルサイズ：128 B

2つのプロセス、1つの物理フレーム



System V 共有メモリ：ftok → shmget → shmat

C tds/td5/shm_counter.c

```
key_t key = ftok(KEYFILE, KEYID);
exit_if(key == -1, "ftok");
int id = shmget(key, SIZE, IPC_CREAT | 0644);
exit_if(id == -1, "shmget");

if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl I");
    printf("segment %d removed\n", id);
    return EXIT_SUCCESS;
}

int *counter = shmat(id, NULL, 0);
exit_if(counter == (void *)-1, "shmat");
counter[0]++; /* a fresh
printf("segment %d (key 0x%08x): counter = %d\n", id, (
exit_if(shmdt(counter) == -1, "shmdt");
```

実際の出力

```
$ ./shm_counter; ./shm_counter; ./shm_counter;
./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

- セグメントは shmctl(IPC_RMID) まで残る。
ipcs -m で一覧
- 各プロセスは別のアドレスで見ている：
ポインタは決して保存しない
- IPC_PRIVATE：キーなし・SHM_RDONLY：読み取り専用・shmctl：IPC_STAT 読む、IPC_SET 権限変更、IPC_RMID 破棄

メモリの共有には同期が必要

アトミックな操作なし

```
( slot = seg[0] + 1; seg[0] = slot; ) :
```

実際の出力

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

__atomic_add_fetch あり：

実際の出力

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566
357567 357572 357568 357571 357569 357565 357574 357573
357570 357576 357578 357575 357577 357579
```

ユニット7のテーマ：競合状態、ミューテックス、セマフォ。

解答付き TD5 + AI チャレンジ

- `sigsuspend` を使う `mysignal`、送り手と受け手
- `mmap_limits` と 5 つの保護
- `shm_counter`、`shm_fork`、`shm_race` の競合

Bash

```
make -C tds && tds/tests/run.sh td5
```



Universidad
Indoamérica

まとめ

3つの問い

1. ブロック中の SIGUSR1 3 つでハンドラが1回しか実行されないのはなぜ？
2. `pause` では解けず `sigsuspend` が解く問題は何？
3. 共有メモリにポインタを保存しないのはなぜ？